Introduction to useful Web exploitation concepts for solving UniWA-CTF

Michael Kokkos, Alexander Soloviev

Athens, Greece

Abstract

Today's society is heavily reliant on Web services and the Internet in general. As the importance of the Internet grows, malicious activity is growing too. That makes the field of Web application security more relevant than ever, as a seemingly small bug in a web service can cost a lot of resources or even lives. Penetration testing and code analysis are both viable techniques of securing web applications, with each of them taking a different approach on the problem. This paper introduces to the reader some important concepts and assist the participants in the UniWA-CTF tackle not only the stand-alone Web exploitation challenges, but also the Web exploitation problems that appear in the boot-to-root boxes.

Keywords: Security, Web application, Malicious activity, Penetration testing, Code analysis

1. Introduction

In recent years, following the vast expansion of the web trend, cybersecurity and cybercrime have been the major issues and threats for financial institutions, governments, individuals and other stakeholders globally [1].

Almost 500 million consumers have ever been the victim of a cyber crime, with nearly 350 million of them in the 2019 alone. According to 2019 NortonLifeLock Cyber Safety Insights Report, and based on data collected from ten countries [2].

Among the most common types of cyber attacks is the SQL injection [3]. Although it is an old type of attack, with the first public discussions started appearing around 1998 [4], it is still an insisting threat.

Mitigation of these threats requires a certain level of familiarity with concepts of cyber security from the developers, but also requires field experts so that they can perform penetration testing and code auditing before a web application goes into production mode.

UniWA-CTF web challenges and boot-to-root boxes introduce the participants into some basic concepts of web exploitation, including SQL injections, static code analysis and provides some basic theoretical knowledge about the functioning of the search engines (specifically about the web crawlers), with the introduction of challenges that require from the user the utilization of the robots exclusion standard.

2. Static Code Analysis

Static analysis of source code is a scalable method for discovery of software faults and security vulnerabilities [5]. It is a "white-box" approach, which means that the developer analyzes the source code without executing it, looking for potential vulnerabilities.

This technique comes in contrast with another well-known technique used by the web developers which is called "Penetration testing" and it's a "black box" approach, which means that the developers stress the application from the point of view of an attacker using various techniques and tools.

An example of static code analysis tool is the "inspector", which is integrated in the majority of Web browsers and it allows the user to view the HTML source code that a website is written with.

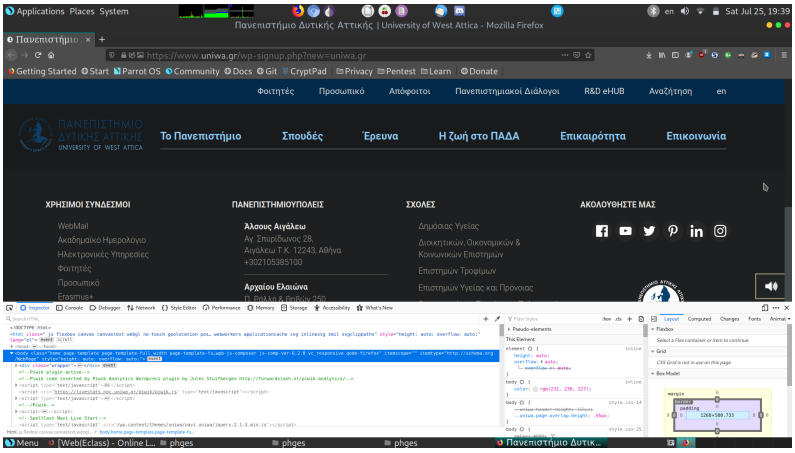

Figure 1: Inspector tool's User Interface in Mozilla Firefox web browser

In the example of Figure 1, it is presented the use of the inspector tool of the Firefox web browser, its use is similar in other web browsers too. It is worth to be mentioned that the environment that includes it is not limited to static code analysis related tools. It provides the user with plenty of useful web developer related tools, such as a JavaScript console, information about outgoing and incoming files and their protocols, cookies, et al.

3. Robots Exclusion Protocol

The robots exclusion protocol or simply robots.txt, is an unofficial standard which tells web crawlers and other web robots which pages or files the crawler can or can't request from a site [6].

3.1. Web crawlers

Web crawlers are software applications designed for exploring web applications automatically [7]. They are usually used by search engines so that they can keep their databases updated and help the end-users find their desired information with precision by querying these databases.

There are three main motivations of using the web crawlers [7].

- Content indexing for search engines.
- Automated testing and model checking for web applications.
- Automated security testing and vulnerability assessment.

The process of crawling starts by giving the software a set of seed URLs as an input, then it downloads all the web pages associated with these URLs. After fetching a web page, the URL is removed from the working queue. Lastly, it parses the downloaded page, extracts the linked URLs from it, and adds new URLs to the list of seed URLs. This process, which is described above, continues until all of the contents reachable from seed URLs are reached [7].

To conclude that short definition, a web crawler is a piece of software that targets a particular topic and visits and gathers only relevant web pages associated with these URLs [8].

3.2. The Standard's Use

In 1993 and 1994 there have been occasions where robots have visited web servers where they weren't welcome for various reasons Vikas Rao Vadi [9]. These incidents indicated the need for established mechanisms for web servers to indicate to robots which parts of their server should not be accessed. This standard addresses this need with an operational solution [10].

The method that is used to exclude robots from a server is to create a text file on the server which specifies the files and directories that the server administrator does not want the robots to access. This file must be accessible via HTTP on the local URL "/robots.txt".

This approach was chosen because of its easy implementation on web servers, and a robot can find the access policy with only a single document retrieval. [10]

An example of a robots.txt is given in Figure 2. It is strongly suggested to the reader to study the standard's official page, which is provided in [10].

```
User-agent: *
Disallow: /aFolder
Disallow: /aFile.txt
Disallow: /anotherFile.html
Disallow: /aThirdFile.bin
```

Figure 2: A robots.txt example

In the Figure 2 example, the robots.txt uses the "*" wildcard to apply its rules to all the robots, it is not required to apply the rules to all the robots, different rules can apply to different robots and the name of the corresponding robot could be used in the "User-agent" field.

The rules can be easily understood without an extended definition. In short, the above example shows, a directory, and three files of different formats (one text file, one Hypertext Markup Language file and one binary file) are disallowed from use by the web crawlers. Disallowing web crawlers from accessing directories and files can be done using the keyword "Dissallow:", followed by the path of the directory or file in the web server.

4. SQL Injection Attacks

SQL injection is a code injection technique. In its simplest form it is a relatively easy technique to perform, even for a person without the proper technical background. An input string is injected through the application to change or manipulate the SQL statement so that the attacker can take advantage. This attack can harm the database in several ways, including unauthorized access and manipulation of the database, and disclosure of sensitive data [11].

Different implementations of SQL injections exist, some of them are so simple that they can be successful just by copying and paste available payloads that can be found on the Internet, and other require in depth knowledge of the SQL language and databases in general.

```
or 1=1
```

Figure 3: An example of an SQL payload

In the example in Figure 3, is depicted a typical SQL payload in its simplest form. If an attacker fill a vulnerable web form with that, he or she "tricks" the database into giving him or her access. That is because the attacker manipulates the query to always return "TRUE" making the intended first argument (that requires the password to be "TRUE") irrelevant, because the second argument states that one is equal to one which is always "TRUE".

Explaining all the implementations of the attack and its countermeasures would be out of the scope of this paper, however there is a great paper that does so, it is provided in [12], and the participants are advised to study it if they want to deepen their knowledge about these kinds of attacks.

An SQL injection vulnerability can be discovered either by performing code analysis (static or dynamic), or either by trial and error by utilizing different payloads.

5. Conclusions

Web application penetration testing skills are useful not only to information security engineers or bounty hunters. A basic knowledge of web application security concepts, such as these that were described in this paper, can apply to subjects such as these of software and web application development. The stand-alone and boot-to-root web challenges of UniWA-CTF have been designed with that in mind, so that the participation can have a positive effect to a wide range of students.

References

[1] F. Kamy, S. Conrad, V. Jay, Cybersecurity indices and cybercrime annual loss and economic impacts, Journal of Business and Behavioral Sciences 32 (2020) 63–71.

[2] NortonLifeLock Inc., 2020, 2019 Cyber safety insights report global results, URL: https://us.norton.com/nortonlifelock-cyber-safety-report.

[3] Rapid7, 2020, Common types of cybersecurity attacks, URL: https://www.rapid7.com/fundamentals/types-of-attacks/.

[4] S. M. Kerner, 2013, How was sql injection discovered?, URL: https://www.esecurityplanet.com/network-security/how-was-sql-injection-discovered.html.

[5] K. Goseva-Popstojanova, A. Perhinschi, On the capability of static code analysis to detect security vulnerabilities, Information and Software Technology 68 (2015) 18 – 33. URL: http://www.sciencedirect.com/science/article/pii/S0950584915001366. doi:https://doi.org/10.1016/j.infsof.2015.08.002.

[6] Google, 2020, Introduction to robots.txt, URL: https://support.google.com/webmasters/answer/6062608?hl=en.

[7] S. Mirtaheri, M. Dincturk, S. Hooshmand, G. Bochmann, G.-V. Jourdan, I.-V. Onut, A brief history of web crawlers, Proceedings of the 2013 Conference of the Center for Advanced Studies on Collaborative Research (2013) 40–54.

[8] P. Gupta, K. Johari, Implementation of web crawler, in: 2009 Second International Conference on Emerging Trends in Engineering Technology, 2009, pp. 838–843. doi:10.1109/ICETET.2009.124.

[9] A. T. Vikas Rao Vadi, Crawler and url retrieving queuing, International Journal of Computer Application 1 (2013) 14.

[10] M. Koster, 1994, A standard for robot exclusion, URL: https://www.robotstxt.org/orig.html.

[11] M. Humayun, M. Niazi, N. Zaman, M. Alshayeb, S. Mahmood, Cyber security threats and vulnerabilities: A systematic mapping study, Arabian Journal for Science and Engineering 45 (2020). doi:10.1007/s13369-019-04319-2.

[12] W. Halfond, J. Viegas, A. Orso, A classification of sql injection attacks and countermeasures (2006).